

Lab Manual

Experiment 1:

Part A

Exploring AWS CloudShell and the AWS Cloud9 IDE

Step 1: Login to AWS Console

1. Open browser and go to:
 <https://aws.amazon.com>
2. Click Sign In to the Console
3. Enter your AWS account credentials

Step 2: Open AWS CloudShell

1. In the top-right corner of AWS Console
2. Click the CloudShell icon (>_)
3. Wait for the terminal to initialize

Step 3: Basic CloudShell Commands

Type the following commands one by one:

- `pwd`
- `ls`
- `whoami`
- `date`

Step 4: Create and Run a Sample File

- `nano hello.sh`
- Type:
- `echo "Hello from AWS CloudShell"`

Save the file (CTRL + O, Enter, CTRL + X)

Run:

```
bash hello.sh
```

Step 5: Check AWS CLI

```
aws --version
```

Experiment 1:

Part B

Creating a Virtual Machine with specific CPU and memory

Step 1: Login to AWS Console

1. Open browser → <https://aws.amazon.com>
2. Click Sign In to the Console
3. Login with AWS credentials

Step 2: Open EC2 Dashboard

1. In the AWS search bar, type EC2
2. Click EC2
3. You will see the EC2 Dashboard

Step 3: Launch a New Instance

1. Click Launch instance
2. Name the instance:
3. VM-Lab-Instance

Step 4: Choose an Amazon Machine Image (AMI)

1. Select:
 - ☒ Amazon Linux 2023
2. This acts as the Operating System

Step 5: Select Instance Type (CPU & Memory)

Choose **Free Tier eligible** instance:

Instance Type	vCPU	Memory
t2.micro	1	1 GB
t3.micro	2	1 GB

Step 6: Create a Key Pair

1. Click Create new key pair
2. Key pair name: vm-lab-key
3. Key pair type: RSA
4. File format: .pem

5. Click Create key pair

Step 7: Configure Network Settings

1. Allow:
 - ✓ SSH (Port 22)
2. Source:
 - Anywhere (for lab purpose only)

Step 8: Configure Storage

- Default: 8 GB gp3
- Leave as is (Free Tier eligible)

Step 9: Launch Instance

1. Review settings
2. Click Launch instance

 **Instance will start launching.**

Step 10: View Instance Status

1. Click View all instances
2. Instance state should show:
 - Running
3. Status check: 2/2 checks passed

Experiment 2:

Working with Amazon S3 Orchestrating Serverless Functions with AWS Step Functions

Procedure

Step 1: Login to AWS Console

1. Open browser and go to: <https://aws.amazon.com>
2. Click Sign In to the Console
3. Login using your AWS credentials

Step 2: Open Amazon S3

1. In the AWS Console search bar, type S3
2. Click Amazon S3
3. You will see the Buckets page

Step 3: Create a New Bucket

1. Click Create bucket
2. Bucket name:
 - Enter a globally unique name
 - Example: student-s3-lab-2025-001
3. AWS Region:
 - Select Asia Pacific (Mumbai) or default region
4. Object Ownership:
 - Keep default (ACLs disabled)
5. Block Public Access:
 - ✓ Keep Block all public access enabled
6. Leave other settings as default
7. Click Create bucket

 **Bucket created successfully.**

Step 4: Open the Bucket

1. Click on the bucket name
2. You will see an empty bucket

Step 5: Upload a File

1. Click Upload
2. Click Add files
3. Select any text file or image from your computer
4. Click Upload

 **File uploaded successfully.**

Step 6: View the Uploaded Object

1. Click the uploaded file name
2. View:
 - Object size
 - Storage class

- Last modified date

Step 7: Download the Object

1. Select the file
2. Click Download
3. File will be saved to your local system

Step 8: Delete the Object

1. Select the uploaded file
2. Click Delete
3. Type delete to confirm
4. Click Delete objects

Step 9: Delete the Bucket Go back to Buckets

1. Select the bucket
2. Click Delete
3. Type the bucket name to confirm
4. Click Delete bucket

Experiment 3:

Working with Amazon DynamoDB

Step 1: Login to AWS Console

1. Open browser and go to: <https://aws.amazon.com>
2. Click Sign In to the Console
3. Login using AWS credentials

Step 2: Open Amazon DynamoDB

1. In the AWS Console search bar, type DynamoDB
2. Click Amazon DynamoDB
3. DynamoDB dashboard will open

Step 3: Create a DynamoDB Table

1. Click Create table
2. Table name: StudentTable
3. Partition key: StudentID (String)
4. Leave Sort key unchecked
5. Table settings:
 - Select Default settings
6. Click Create table

✅ Table will be created successfully.

Step 4: Open the Table

1. Click on StudentTable
2. Go to the Explore table items tab

Step 5: Add an Item to the Table

1. Click Create item
2. Enter details:

Attribute	Value
StudentID	S001
Name	Ramesh

Department	CSE
Semester	6

3. Click Create item

 **Item added successfully.**

Step 6: View Table Items

- All inserted items will be displayed in the table
- Click any item to view its attributes

Step 7: Edit an Item

1. Select an item
2. Click Edit
3. Modify any attribute (e.g., Semester → 7)
4. Click Save changes

Step 8: Delete an Item

1. Select the item
2. Click Delete
3. Confirm deletion

Step 9: Delete the Table (IMPORTANT)

1. Go to Tables
2. Select StudentTable
3. Click Delete
4. Type delete
5. Click Delete table

Experiment 4:

Developing REST APIs with Amazon API Gateway

Step 1: Login to AWS

1. Open a web browser
2. Go to: <https://aws.amazon.com>
3. Click Sign in to the Console
4. Login using AWS account credentials

Step 2: Open Amazon API Gateway

1. In the AWS search bar, type API Gateway
2. Click Amazon API Gateway

Step 3: Create a REST API

1. Click Create API
2. Under REST API, click Build
3. On the *Create REST API* page:
 - Select New API
 - API Name: StudentRESTAPI
 - Endpoint Type: Regional
4. Click Create API

Step 4: Create a Resource

1. Click Resources
2. Select the root resource /
3. Click Create resource
4. Enter:
 - Resource name:
 - hello
 - Resource path:
 - /hello
5. Click Create resource

Step 5: Create a GET Method

1. Select the /hello resource
2. Click Create method
3. Select GET
4. Click Create method

Step 6: Configure Integration Type

1. In Integration type, select:
2. MOCK
3. Authorization: NONE
4. Click Save

Step 7: Configure Method Response

1. Click Method response
2. Click Add response
3. Enter:
4. 200
5. Click Save

Step 8: Configure Integration Response

1. Click Integration response
2. Click Edit
3. Under Mapping templates:
 - Content-Type: application/json
 - Select Empty
4. In the response body editor, enter:

```
{  
  "message": "Hello from Student REST API"  
}
```

5. Click Save

Step 9: Deploy the API

1. Click Deploy API
2. Stage name:
3. dev
4. Click Deploy

Step 10: Test the API

1. Click Stages
2. Select dev
3. Copy the Invoke URL
4. Append /hello to the URL

Example:

`https://xxxx.execute-api.us-east-1.amazonaws.com/dev/hello`

5. Paste the URL into a browser and press Enter

Expected Output

```
{  
  "message": "Hello from Student REST API"  
}
```

Experiment 5:

Creating Lambda Functions Using the AWS SDK for Python

Step 1: Login to AWS

1. Open a browser
2. Go to: <https://aws.amazon.com>
3. Click Sign in to the Console
4. Login using AWS credentials

Step 2: Open AWS Lambda

1. In the AWS search bar, type Lambda
2. Click AWS Lambda

Step 3: Create a Lambda Function

1. Click Create function
2. Select Author from scratch
3. Enter:
 - Function name: HelloLambdaFunction
 - Runtime: Python 3.14
4. Under Permissions:
 - Choose Create a new role with basic Lambda permissions
5. Click Create function

Lambda function is created.

Step 4: Write Python Code (Using AWS SDK – boto3)

```
import json
```

```
import boto3
```

```
def lambda_handler(event, context):
```

```
    return {
```

```
        'statusCode': 200,
```

```
        'body': json.dumps('Hello from AWS Lambda using Python SDK')
```

```
    }
```

Click Deploy

Step 5: Test the Lambda Function

1. Click Test
2. Choose Create new test event
3. Event name: TestEvent
4. Leave default JSON unchanged
5. Click Save
6. Click Test

Expected Output

```
{  
  "statusCode": 200,  
  "body": "\"Hello from AWS Lambda using Python SDK\""  
}
```

Experiment 6:

Migrating a Web Application to Docker Containers

Step 1: Login to AWS

1. Go to <https://aws.amazon.com>
2. Click Sign in to the Console
3. Login with AWS credentials

Step 2: Launch EC2 Instance

1. Open EC2
2. Click Launch instance
3. Configure:
 - Name: Docker-WebApp
 - AMI: Amazon Linux
 - Instance type: t2.micro
 - Security Group:
 - Allow SSH (22)
 - Allow HTTP (80)
4. Click Launch instance

Step 3: Connect to EC2

1. Select instance → Connect
2. Choose EC2 Instance Connect
3. Click Connect

Step 4: Install Docker

Run commands one by one:

```
sudo yum update -y
```

```
sudo yum install docker -y
```

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

```
sudo usermod -aG docker ec2-user
```

Logout and reconnect to EC2.

Step 5: Create Project Directory

`mkdir webpage`

`cd webpage`

Step 6: Create HTML File using vi

`vi index.html`

In vi editor:

1. Press i (insert mode)
2. Type the following code:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>docker web page</title>
```

```
</head>
```

```
<body>
```

```
  <h1>hii welcome to webpage</h1>
```

```
</body>
```

```
</html>
```

3. Press Esc
4. Type: `:wq`
5. Press Enter

Step 7: Create Dockerfile using vi (VERY IMPORTANT)

`vi Dockerfile`

In vi editor:

1. Press i
2. Type: FROM nginx:latest

```
  COPY index.html /usr/share/nginx/html/index.html
```

3. Press Esc
4. Type: **`:wq`**
5. Press Enter

Step 8: Verify Files

ls

You MUST see:

Dockerfile

index.html

Step 9: Build Docker Image

docker build -t aws-docker-webpage .

Expected message:

Successfully built

Successfully tagged aws-docker-webpage:latest

Step 10: Verify Docker Image

docker images

Expected: aws-docker-webpage latest

Step 11: Run Docker Container

docker run -d -p 80:80 aws-docker-webpage

Step 12: Verify Container

docker ps

You should see: 0.0.0.0:80->80/tcp

Step 13: Test Inside EC2

curl http://localhost

HTML output should appear.

Step 14: Access from Browser

1. Go to EC2 Dashboard
2. Copy Public IPv4 address
3. Open browser: http://<PUBLIC-IP>

Expected Output

hii welcome to webpage

Experiment 7:

Caching Application Data with ElastiCache, Caching with Amazon CloudFront, Caching Strategies

PART A: Caching Application Data with Amazon ElastiCache (Redis)

Step 1: Login to AWS

1. Open browser → <https://aws.amazon.com>
2. Click Sign in to the Console
3. Login with AWS credentials

Step 2: Open Amazon ElastiCache

1. Search ElastiCache in AWS console
2. Click Amazon ElastiCache
3. Click Caches → Create cache

Step 3: Cache Settings (IMPORTANT)

Select the following exactly:

- Engine: Redis OSS
- Deployment option: Node-based cluster
- Creation method: Easy create
- Configuration: Demo

Step 4: Cache Name

Enter: student-redis-cache

Step 5: Network & Security

- Keep Default VPC
- Keep Default Subnet
- Do NOT enable public access
- Keep all other settings default

Step 6: Create Cache

1. Scroll down
2. Click Create cache
3. Wait until status becomes Available

PART B: Caching with Amazon CloudFront

Step 7: Create S3 Bucket (Origin)

1. Search S3
2. Click Create bucket
3. Bucket name: student-cloudfront-cache
4. Uncheck Block all public access
5. Click Create bucket

Step 8: Upload Static Content

Upload a file named index.html with content:

```
<!DOCTYPE html>

<html>

<head>

  <title>docker web page</title>

</head>

<body>

  <h1>hii welcome to webpage</h1>

</body>

</html>
```

Step 9: Enable Static Website Hosting

1. Open the bucket
2. Go to Properties
3. Enable Static website hosting
4. Index document: index.html

Step 10: Create CloudFront Distribution

1. Search CloudFront
2. Click Create distribution
3. Origin domain → Select your S3 bucket
4. Viewer protocol policy → Redirect HTTP to HTTPS
5. Click Create distribution

Step 11: Access Cached Content

1. Wait until status is Deployed
2. Copy CloudFront domain name
3. Open in browser: `https://<cloudfront-domain>`

Experiment 8:

Implementing CloudFront for Caching and Application Security

Create a file `index.html` with the given code:

```
<!DOCTYPE html>

<html>

<head>

  <title>docker web page</title>

</head>

<body>

  <h1>hii welcome to webpage</h1>

</body>

</html>
```

STEP 1 Create S3 Bucket

1. AWS Console → S3
2. Click Create bucket
3. Bucket name: `cloudfront-secure-demo`
4. Block all public access → ☒ ON
5. Click Create bucket

Upload File

1. Open the bucket
2. Click Upload
3. Upload `index.html`
4. Click Upload

STEP 2 Create CloudFront Distribution

1. AWS Console → CloudFront
2. Click Create distribution

Origin settings:

- Origin domain → select:
- cloudfront-secure-demo.s3.amazonaws.com
- Origin access → Origin access control (OAC)
- Save

Default behavior:

- Viewer protocol policy → Redirect HTTP to HTTPS
- Cache policy → CachingOptimized

Click Create distribution

Wait until it get deployed

STEP 3 Fix XML Error

1. CloudFront → Distribution → General
2. Click Edit
3. Default root object:
4. index.html
5. Save

STEP 4 Create Security Headers

1. CloudFront → Policies
2. Click Response headers policies
3. Click Create

Fill:

- Name: security-headers

Enable:

- ✓ X-Content-Type-Options → nosniff
- ✓ X-Frame-Options → DENY
- ✓ Strict-Transport-Security → Enabled

Click Create

STEP 5 Attach Security Headers

1. CloudFront → Distribution → Behaviors
2. Select Default behavior
3. Click Edit
4. Response headers policy → security-headers
5. Save

STEP 6 Allow CloudFront to Read S3

1. S3 → cloudfront-secure-demo
2. Go to Permissions
3. Click Bucket policy
4. Paste this 🖱️

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::cloudfront-secure-demo/*",
      "Condition": {
        "StringEquals": {
          "AWS:SourceArn": "arn:aws:cloudfront::ACCOUNT-
ID:distribution/DISTRIBUTION-ID"
        }
      }
    }
  ]
}
```

}

👉 **Replace:**

- **ACCOUNT-ID**
- **DISTRIBUTION-ID**

Click Save

STEP 7 Test

Open browser:

<https://<your-cloudfront-domain>.cloudfront.net>

Experiment 9:

Orchestrating Serverless Functions with AWS Step Functions

Step 1: Login to AWS

1. Open browser
2. Go to <https://aws.amazon.com>
3. Click Sign in to the Console
4. Login using AWS credentials

Step 2: Open AWS Step Functions

1. In the AWS search bar, type Step Functions
2. Click AWS Step Functions
3. Click State machines
4. Click Create state machine

Step 3: Create State Machine (IMPORTANT SCREEN)

On the Create state machine screen:

1. Select:
 - ☒ Create from blank
2. State machine name:
3. StudentStepFunction
4. State machine type:
 - ☒ Standard
5. Click Continue

→ This opens Workflow Studio

Step 4: Add FIRST Lambda Function

1. On the left panel, under MOST POPULAR
2. Drag AWS Lambda – Invoke
3. Drop it in the dotted area

Now click the Lambda box and set (right panel):

- State name: FirstStep
- Lambda function: FirstFunction

Step 5: Add SECOND Lambda Function (MOST MISSED STEP)

1. Again, drag AWS Lambda – Invoke
2. Drop it below FirstStep

Click this second Lambda and set:

- State name: SecondStep
- Lambda function: SecondFunction

✓ Final workflow must look like:

Start → FirstStep → SecondStep (End)

Step 7: Create the State Machine

1. Click Create (top-right)
2. IAM Role: Select Create new role
3. Click Create state machine

Wait for success message

Step 8: Execute the Workflow

1. Click Start execution
2. Input:

```
{  
  "message": "Hello Step Functions"  
}
```

3. Click Start execution

Step 9: Observe Execution

- FirstStep turns green
- SecondStep turns green
- Execution status: Succeeded

This confirms orchestration worked correctly

Experiment 10:

Automating Application Deployment Using a CI/CD Pipeline

STEP 1: CREATE GITHUB REPOSITORY

1. Open **GitHub**
2. Create a **public repository** named:
3. Automating-Application-Deployment-Using-a-CI-CD-Pipeline
4. Create a file called **index.html**
5. Paste this code and save:

```
<!DOCTYPE html>

<html>

<head>

  <title>CI/CD Demo</title>

</head>

<body>

  <h1>CI/CD Deployment Successful</h1>

  <p>Deployed automatically using EC2 User Data</p>

</body>

</html>
```

STEP 2: LAUNCH EC2 INSTANCE

1. Login to **AWS Console**
2. Go to **EC2** → **Launch instance**
3. Instance name: CICD-EC2-Server
4. AMI: Amazon Linux 2023
5. Instance type: t2.micro or t3.micro
6. Select or create a **key pair**

STEP 3: ADD USER DATA (CI/CD SCRIPT)

1. Scroll down to **Advanced details**
2. In **User data**, paste this **exact code**:

```
#!/bin/bash
```



```
yum update -y
```

```
yum install -y httpd git
```

```
systemctl start httpd
```

```
systemctl enable httpd
```

```
cd /var/www/html
```

```
rm -rf *
```

```
git clone https://github.com/<github -hub -user-name>/repositoryname.git (Change Accordingly)
```

```
cp -r Github repo name/* .
```

STEP 4: LAUNCH INSTANCE

Click **Launch instance**

Wait **2–3 minutes**

STEP 5: ACCESS THE WEBSITE

1. Go to **EC2 → Instances**
2. Copy **Public IPv4 address**
3. Open browser and type:

http://<Public-IP>

Output: CI/CD Deployment Successful

STEP 6: VERIFY CI/CD

1. Open index.html in GitHub
2. Change the text
3. Save (commit)
4. **Reboot EC2 instance**
5. Refresh browser